Computer Vision — Teaching Notes (for the exam)

Use this with STUDY-ORDER.md.
That file is **what to open, in which order**. This file is **the actual teaching**.

Read one section. Close the laptop. Say it out loud in your own words. If you cannot, read that section again. That is how you pass a theory paper.

The exam wants **definitions, why it exists, how it works, compare two things, give one example**. Code is extra.

How to read this

Each topic has the same shape:

1. **The idea** — what it is, like I am telling you in class
2. **Why we need it**
3. **How it works**
4. **Write this in the paper** — a ready answer in simple English

Do Step 1 first. Do not jump to YOLO.

STEP 1 — Image foundations (Week 1)

This is Module 1 and the start of Module 2. If this is weak, later CNN answers will sound empty.

---

1. What is Computer Vision?

A computer does not "see" a photo the way you do. You see a face, a car, a tree. The computer only sees **numbers**.

Computer Vision is the part of AI that turns those numbers into meaning: "this is a face", "the car is here", "this pixel is road".

Used in: self-driving cars, medical scans, CCTV, phone unlock, AR filters.

Write this in the paper:
Computer Vision is a field of AI that enables machines to understand visual information from images or video. A digital image is stored as numbers. CV algorithms process those numbers to do tasks like classification, detection and segmentation.

2. An image is a matrix of pixels

Imagine a chessboard. Each square is one **pixel** (picture element). Each pixel holds a number for brightness or color.

Grayscale (black and white)

One number per pixel, usually **0 to 255**.

- `0` = black
- `255` = white
- `128` = gray

So a grayscale image is a **2D matrix**: rows × columns.
A 3×3 grayscale image can look like:

```
  0   128  255
 64   192  128
255   255    0
```

Shape: `(height, width)` example `(5472, 3648)`.

Color

A color pixel needs **three** numbers: Red, Green, Blue.
So a color image is a **3D matrix**: height × width × **3 channels**.

Shape: `(height, width, 3)`.

Pure red in RGB is `[255, 0, 0]` (red full, green off, blue off) and similar for green and blue.

Where is (0, 0)?

In maths, origin is often bottom-left. **In images, origin (0, 0) is the top-left.**

Mathematics Coordinate System



- The origin **(0,0)** is at the **bottom-left**.

- **Bottom-left** only tells you **where the origin (starting point)** is. it is shown when you put your eyes at dot(no-concept just menomics).
- It **does NOT** tell you the direction of the x-axis.
- From the origin:
- Move **right** → **+x**
- Move **up** → **+y**

Image Coordinate System

```
(0,0) ──┼──────────→ +x
        |
        |
        |
        ↓        .
       +y
```

- The origin **(0,0)** is at the **top-left**.
- **Top-left** only tells you **where the origin (starting point)** is. it is shown when you put your eyes at dot.(no-concept just menomics).
- It has their own mathmatics system.
- Images use their **own coordinate system**:
- Move **right** → **+x**
- Move **down** → **+y**

Remember for the Exam

- **"Bottom-left"** and **"Top-left"** describe **where (0,0) is located**, **not** the direction of the axes.
- The **x-axis always increases to the right**.
- The only difference is the **y-axis**:
- **Mathematics:** +y is **up**.
- **Images:** +y is **down**.
- `x` grows to the **right**
- `y` grows **down**

To read a pixel you use `image[y, x]` — row first, then column. That is why `image[5, 10]` is "5 down, 10 across", not the other way.

Why top-left? Screens and files are stored row by row starting from the first row at the top. 👈 👈 👈

OpenCV vs Matplotlib (very common viva)

What is OpenCV?

OpenCV (Open Source Computer Vision Library) is a **software library** that helps computers **read, understand, process, and analyze images and videos**.

It provides **thousands of ready-made functions**, so you don't have to write image-processing algorithms from scratch.

Features:

- Open source (free to use)
- Cross-platform (Windows, Linux, macOS)
- Supports Python, C++, Java, and other languages

What does OpenCV do?

Think of **OpenCV as a toolbox for images and videos**.

Just as **NumPy** provides ready-made functions for working with **arrays**, **OpenCV** provides ready-made functions for working with **images and videos**.

For example, with a single OpenCV function, you can:

- Read an image
- Convert it to grayscale
- Resize it
- Detect edges
- Detect faces

Instead of writing these algorithms yourself, you simply call OpenCV's built-in functions.

- **OpenCV** stores color as **BGR** (Blue, Green, Red).
- **Matplotlib** (and most websites) use **RGB**.

Same three numbers, different order. If you display an OpenCV image in Matplotlib without converting, red and blue look swapped.

Write this in the paper:
A grayscale image is a 2D matrix of intensities 0–255. A color image is a 3D matrix with 3 channels. Origin is at the top-left. OpenCV uses BGR; Matplotlib uses RGB.

---

3. Color spaces: RGB vs HSV

RGB is how screens mix light: turn red, green, blue lamps up or down. Fine for display. Bad for "find all yellow fruit", because yellow in shade and yellow in sun have very different RGB numbers.

Explanation

- **RGB** creates colors by mixing different amounts of **Red, Green, and Blue** light. Every pixel on a screen uses these three colors to display any color.
- It is **excellent for displaying images** because screens naturally emit red, green, and blue light.
- However, RGB is **not reliable for color detection** because the RGB values of the **same object change with lighting**.
- For example, a yellow banana in **bright sunlight [255, 255, 50]** and the **same banana in shade [120, 120, 20]** are both yellow to our eyes, but their RGB values can be very different.
- Therefore, if we search for a fixed RGB value to detect yellow objects, many yellow objects may be missed and create a confusion.

Exam Tip: RGB is best for **displaying colors**, but not ideal for **detecting colors under different lighting conditions**.

HSV splits color the way humans talk about it:

Letter	Meaning	Think of it as
H Hue	Which color family	the angle on a color wheel (0° red, 120° green, 240° blue)
S Saturation	How pure / washed	0% = gray, 100% = vivid
V Value	How bright (Intensity)n.	0% = black, 100% = bright

Why do we need HSV?

RGB is excellent for **displaying colors**, but it is **not reliable for color detection**.

For example, suppose you want to detect a **yellow banana**.

- In bright sunlight, its RGB values might be: `text (255, 255, 50)`

- In shade, the **same banana** might have: `text (120, 120, 20)`

Although the banana is still **yellow**, its RGB values change a lot because the lighting changes. Therefore, detecting colors using RGB is difficult.

How does HSV solve this problem?

HSV separates **color** from **brightness**.

Instead of storing only Red, Green, and Blue, it stores three different properties that is Hue, saturaton and value and each represent different things.

Example

Banana in bright sunlight

```
Hue = Yellow
Saturation = High
Value = High (Bright)
```

Same banana in shade

```
Hue = Yellow
Saturation = High
Value = Low (Dark)
```

What changed?

- ✅ Hue stayed **Yellow**.
- ✅ Saturation stayed almost the same.
- ✅ Only Value (brightness) changed.

Since the **Hue remains almost the same**, the computer can still recognize the banana as **yellow**, even when the lighting changes.

Why is HSV better than RGB for color detection?

HSV separates the **color** from the **brightness**.

When lighting changes, **Value (brightness)** changes the most, while **Hue (the actual color)** remains nearly the same. Therefore, the computer can detect the same color under different lighting conditions more reliably than with RGB.

Exam Definition

> **HSV (Hue, Saturation, Value) is a color model that separates color from brightness. Hue represents the color, Saturation represents the purity of the color, and Value represents the brightness. HSV is preferred for color than RGB detection because changes in lighting mainly affect the Value, while the Hue remains relatively stable.**

---

4. Image transformations

Sometimes you do not change *what* is in the picture. You only **move, resize, or warp** it. That is a geometric transform.

Name	What it does	Everyday example
Translation	Shift left/right/up/down	sliding a photo on a table
Scaling	Make bigger or smaller	zoom
Rotation	Turn around a point	rotate 45° around the center
Affine	Stretch / shear, but **parallel lines stay parallel**	a rectangle can become a parallelogram
Perspective	Perspective Transformation changes the viewing angle of an image. It is mainly used to remove perspective distortion, such as converting a tilted document into a flat, scanned-like image.	document scanner: a tilted page becomes a flat rectangle

Example for perspective:

Before (Tilted (Quad)) After (Flat (rectangle))

_ _ _ __

/ / | |

/ / -> | |

// /__|

Affine needs **3 point pairs**. Perspective needs **4 point pairs** (the four corners of a page).

These are also used as **data augmentation** when training a CNN (random rotate/scale so the model does not memorize one pose).

Write this in the paper:
Translation shifts the image, scaling resizes it, rotation turns it. Affine preserves parallelism. Perspective maps a quadrilateral to another and is used in document scanning.

---

5. Kernel (filter) — the most important small idea

A kernel (filter or convolution matrix) is a small matrix of numbers (usually 3×3 or 5×5) that slides over an image. At each position, it combines the center pixel and its neighboring pixels to calculate a new pixel value. Different kernels perform different operations such as blurring, sharpening, and edge detection.

A **kernel** is a tiny grid of numbers, usually 3×3 or 5×5. Other names: filter, mask, convolution matrix.

You place it on a pixel and its neighbors, multiply matching cells, **add them up**. That sum becomes the **new** pixel.

Same image + different kernel = completely different result:

Kernel job	What it does
Identity	leave the image as is
Box / average blur	mix neighbors → smooth, less noise
Gaussian blur	same idea, but center counts more than far neighbors
Sharpen	boost the center, subtract neighbors → edges look crisper
Sobel X	highlight **vertical** edges
Sobel Y	highlight **horizontal** edges

Larger kernel (7×7 vs 3×3) looks at a **wider** neighborhood: stronger blur, slower, more context, less tiny detail.

In **traditional image processing**, we manually design the kernel values for a specific task, such as blurring, sharpening, or edge detection. In a **Convolutional Neural Network (CNN)**, we still use kernels, but **we do not choose their values ourselves**. Instead, the CNN **learns the best kernel values automatically from training data**. During training, it adjusts the kernel values to detect useful patterns such as edges, textures, shapes, and eventually complex objects like faces or cars. This is why CNNs are much more flexible and powerful than manually designed filters.

Write this in the paper:
A kernel is a small matrix that slides over an image. At each position we multiply overlapping values and sum them to get a new pixel. Different kernels produce blur, sharpen or edges. In CNNs, kernels are learned automatically to detect patterns.

---

6. Convolution

Convolution is a mathematical operation where a small matrix (kernel) slides over the image, multiplying and summing pixel values to produce a new output. its purpose is to Extracts features such as edges, textures, or smoothness.

Different kernels do different jobs:

- Blur kernel → smooths the image
- Sharpen kernel → makes edges clearer
- Edge detection kernel → finds boundaries

Convolution is the name of that sliding process.

1. Put the kernel on the top-left of the image.
2. Multiply + add.
3. Store the result in the output.
4. Slide the kernel by a step (that step is later called **stride**).
5. Repeat until the whole image is covered.

The output is a **feature map** — a new image that highlights whatever the kernel cares about (smoothness, edges, ...).

Formula you may be asked to write (2D convolution):

[(I * K)(x, y) = \sum_m \sum_n I(x - m,\ y - n) \cdot K(m, n)]

`I` is the image, `K` is the kernel. Do not panic: it only says “look at neighbors, weight them with K, add”.

Write this in the paper:
Convolution is a sliding-window operation. A kernel moves across the image, computes a weighted sum of local pixels, and produces a feature map. It is the foundation of both classical filtering and CNNs.

---

7. Edges: Sobel and Canny

An **edge** is a place where brightness jumps (the outline of an object).

Sobel is a pair of kernels:

- Sobel X finds vertical edges (left vs right change)
- Sobel Y finds horizontal edges (up vs down change)

Together they give an edge map. Sobel is simple but a bit noisy.

Canny is an advanced edge detector. It also finds edges, but it removes noise and keeps only the **most important, clean edges**.

- Removes noise.
- Finds edges.
- Removes weak or false edges.
- Keeps only strong, connected edges.

Result: cleaner, thinner edges. Used in lane detection, document outlines, medical boundaries.

Write this in the paper:
Edge detection finds pixels where intensity changes sharply. Sobel uses gradient kernels for horizontal and vertical edges. Canny combines smoothing, gradient, non-max suppression and double thresholding, so edges are cleaner.

---

8. Histogram

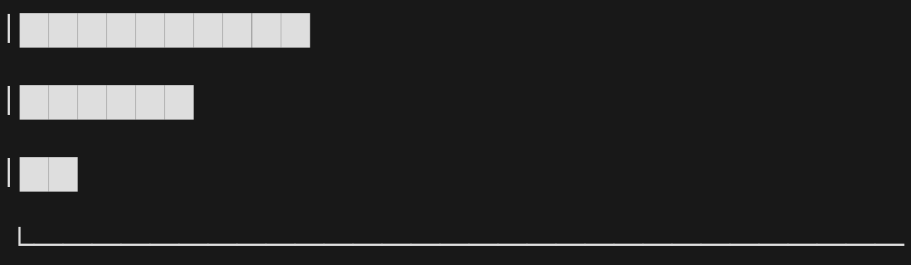
A **histogram** counts how many pixels have each intensity from 0 to 255.

- X-axis: intensity (dark → bright)
- Y-axis: how many pixels

example

Intensity →

0 255



What you can read from it:

- Peak on the **left** → image is dark (underexposed)
- Peak on the **right** → image is bright (overexposed)

- **Narrow** peak → low contrast (washed / foggy)
- **Spread** across 0–255 → good contrast

It does not tell you *what* objects are in the photo. It only tells you about brightness and contrast. That is still enough to decide "should I enhance this?"

Write this in the paper:
A histogram shows the distribution of pixel intensities. It helps judge brightness and contrast. A left-shifted histogram is underexposed; a right-shifted one is overexposed; a narrow histogram means low contrast.

---

9. Histogram equalization (thin in class notes — still asked)

Suppose most pixels sit between 40 and 90. The photo looks dull because it is not using the full 0–255 range.

Histogram equalization stretches those values across 0–255 so darks get darker and brights get brighter. Contrast improves. Details become easier to see. Often done **before** thresholding or feature detection.

Limit: it can also boost **noise**.

Write this in the paper:
Histogram equalization remaps intensities so they use the full 0–255 range, which improves contrast. It is useful as a preprocessing step but can amplify noise.

---

10. Thresholding — turn gray into black and white

Thresholding converts a grayscale image into a black-and-white (binary) image by choosing a cutoff value called the threshold (T).

Process:

- If pixel intensity > threshold(T) → set to white (255).
- If pixel intensity ≤ threshold(T) → set to black (0).

example:

Grayscale pixels:

50 120 180 240

T = 127

After thresholding:

0 0 255 255

This removes gray shades and leaves only **black and white**, making objects easier to separate from the background.

Types of Thresholding

1. Global Thresholding

A **single threshold (T)** is used for the **entire image**.

Example:

```
Whole image → T = 127
```

It works well when the lighting is **uniform**. If one part of the image is bright and another part is dark, one threshold cannot correctly separate the object everywhere.

---

2. Adaptive Thresholding

Instead of using one threshold for the whole image, it **calculates a different threshold for each small region**.

Example:

```
Bright area  → T = 150
Dark area    → T = 80
```

This works well when the image has **uneven lighting**, because each region gets its own suitable threshold.

---

3. Otsu's Method

In Otsu's method, **you do not choose the threshold**.

The computer **automatically finds the best threshold** that separates the image into **foreground (object)** and **background** as clearly as possible. It works best when the histogram has **two distinct peaks**, such as black text on a white page.

Why is Thresholding Used?

Thresholding simplifies an image by removing unnecessary gray levels, making it easier to detect or analyze objects. It is commonly used in **document scanning**, **object detection**, and **medical image analysis**.

Exam Tip

- **Thresholding:** Converts grayscale → black & white using a threshold **T**.
- **Global:** One threshold for the whole image.
- **Adaptive:** Different threshold for different regions.
- **Otsu:** Computer automatically chooses the best threshold.

Write this in the paper:
Thresholding converts grayscale to binary. Global uses one cutoff and fails under uneven light. Adaptive uses local cutoffs. Otsu automatically chooses T by minimizing intra-class variance. Used in document scanning and medical segmentation.

---

11. Morphological operations — edit shapes, not colors

The core idea of **Morphological Operations** is to **change the shape of objects** in an image. (An **object** is **anything you want the computer to detect or analyze**. Eg: In a document image, the **letters** are the objects.)

They are mainly used on **binary (black-and-white) images**, where:

- **White** = Object
- **Black** = Background

To modify the object's shape, morphological operations use a **structuring element (kernel)**, which is a small shape (such as a **3×3 square, circle, or cross**). This structuring element **slides over the entire image** and decides how each pixel should change based on its neighboring pixels. Depending on the operation, it can **expand (grow)** objects, **shrink** objects, **remove small noise**, or **fill small gaps and holes**. Therefore, morphological operations are mainly used to **clean up images and improve the shape of objects** before further processing.

Exam Tip

> **Morphological operations are image processing techniques used to modify the shape of objects in an image. They use a small structuring element (kernel) that slides over the image to expand, shrink, or clean objects, and are mainly applied to binary images.**

Think of a marker and an eraser on a drawing:

Operation	Think of it as	What it does	Example Use
Dilation	**Grow the object**	Makes the white object **bigger**. Closes small gaps and connects broken parts.	Connect broken letters.
Erosion	**Shrink the object**	Makes the white object **smaller**. Removes tiny white dots (noise).	Thinning lines or removing stray dots.
Opening	**Clean the object** *(Erosion → Dilation)*	First removes small noise, then restores the main object.	Clean noisy scanned images.
Closing	**Repair the object** *(Dilation → Erosion)*	First fills small holes or gaps, then restores the object's original size.	Fill holes and reconnect broken characters(broken characters)

⭐⭐⭐

- Erosion followed by dilation
- Dilation followed by erosiom.

⭐⭐⭐

Opening vs closing is a favourite compare question. Remember the **order**.

Write this in the paper:
Morphological operations modify object shape using a structuring element. Dilation expands foreground, erosion shrinks it. Opening = erosion then dilation (remove noise). Closing = dilation then erosion (fill holes).

Applications

• Document processing: Remove noise, enhance text readability.

• Medical imaging: Highlight structures like cells or tissues.

• Industrial inspection: Detect defects in materials.

• Object detection: Clean up shapes before contour analysis.

---

12. Contours and shapes

After thresholding, a white blob has a **boundary**. That boundary curve is a **contour**. (A **contour** is the **outline (boundary)** of an object in an image.)

From a contour you can compute:

- **Bounding box** — smallest upright rectangle around the object (used to *localise* "the object is here")
- **Convex hull** — like putting a rubber band around the shape; no dents. Simplifies the outline.
- **Area, perimeter, centroid** — numbers you can use to classify "big vs small", "round vs long"
- **Polygon approximation** (Douglas–Peucker): simplify the contour to a few corners.
  3 corners → triangle, 4 → rectangle, many → circle.

Write this in the paper:
A contour is a curve joining continuous boundary points of the same intensity. Bounding boxes localise objects. A convex hull is the smallest convex polygon enclosing the contour. Vertex count after approximation can classify shapes.

STEP 2 — Features, motion, face, first CNN (Week 2)

Rest of Module 2, Module 5 (face), start of Module 3 (CNN words).

STUDY-ORDER.md says: first finish the leftover pages of Week 1 KeyConcepts, **in that PDF's order**, then do Week 2. Those leftover PDF topics start below.

---

13. Object Tracking

A photo is one moment. A **video** is many photos in a row. **Object tracking** means: keep finding the **same** object in every next frame — a ball, a car, a person.

Why we need it: CCTV, self-driving cars, sports replay. Detection says "there is a car in this frame". Tracking says "this is the **same** car as 0.1 seconds ago".

Three approaches (from the PDF):

1. **Detection-based tracking** — run a detector (YOLO, Faster R-CNN) on **every** frame, then **link** boxes that belong to the same object. Accurate when the detector is good. Heavy.
2. **Tracking-by-motion** — do not re-detect from scratch. Follow pixels or a box using **optical flow** or a correlation filter. Faster. Can lose the object if it hides or jumps.
3. **Hybrid** — detect sometimes, predict in between. **SORT** and **DeepSORT** do this (detect + predict the next position).

Challenges: occlusion (object hidden), scale change (near/far), many similar objects, messy background.

Applications: surveillance, autonomous driving, sports analytics.

Write this in the paper:
Object tracking continuously locates the same object across video frames. Detection-based tracking detects in each frame and links boxes. Tracking-by-motion follows the object with optical flow. Hybrid methods such as SORT/DeepSORT combine detection and prediction. Challenges include occlusion, scale change and clutter.

---

14. Edge detection in the KeyConcepts PDF (Sobel, Canny, Laplacian)

Def: This sudden change in pixel intensity is called an **edge**. or An edge is the boundary where one object ends and another begins.

You already learned Sobel and Canny in **section 7**. The same PDF adds one more filter and one warning.

Laplacian looks at how fast intensity is **changing in all directions at once** (second derivative). It highlights edges as thin lines, but it is **noisy**, so we usually blur first (Laplacian of Gaussian).

Important exam note from the PDF: edges are distinctive, but they are **not unique landmarks**. If you slide along an edge, every point looks similar, so matching/tracking prefers **corners**, not edges.

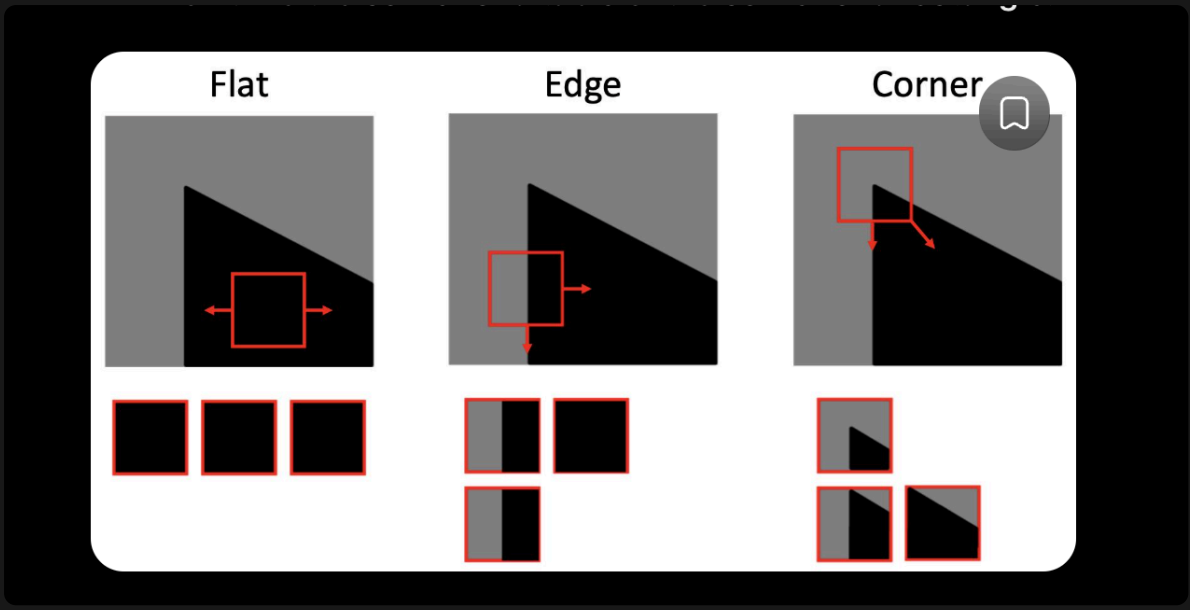
Applications: object segmentation, lane detection, medical imaging.

Write this in the paper:
Edge detection finds sharp intensity changes. Methods include Sobel, Canny and Laplacian. Edges are useful outlines but are not unique landmarks, so corners are better for matching.

---

15. Corner Detection (Harris, FAST, Shi-Tomasi)

A **corner** in an image is a point where the image changes direction sharply or A corner is a pixel where **two or more edges meet**.



Purpose (write this): Find points where edges meet, i.e., regions with intensity variation in multiple directions.

Methods

Method	Simple idea	Exam point
Harris	Give each pixel a "how much is this a corner?" score using how fast brightness changes	Easy. Good on buildings and chessboards. If you zoom or rotate a lot, it can fail
FAST	Look at a ring of pixels around a point. If many are much brighter or much darker, call it a corner	Very fast. Used inside ORB
Shi-Tomasi	Same family as Harris, with a slightly better score	Used when we pick "good points to track" (KLT)

Why useful: corners can be found again in another photo. Flat patches and edges cannot.

Uses: match two photos, track a point in video, stitch a panorama.

Write this in the paper:
Corner detection finds points where edges meet — places where intensity changes in more than one direction. Corners are repeatable and unique, unlike edges. Common methods are Harris, FAST and Shi-Tomasi. Used in matching, tracking and panorama stitching.

---

16. SIFT (Scale-Invariant Feature Transform)

Purpose (write this): Detect and describe key points that still work when the object is bigger/smaller (**scale**) or turned (**rotation**).

In simple words: you take a photo of a book. Later you take another photo closer, or with the book rotated. SIFT still finds the same spots on the cover and can match them.

Pipeline (same 4 steps as the PDF — learn this order):

1. **Gaussian scale space** — make many copies of the image, each a bit more blurry. Blur = looking at the object at a different size.
2. **Difference of Gaussians (DoG)** — subtract one blurry copy from the next. Points that "pop" here are the key points.
3. **Assign orientation** — look at the edge direction around the point. If the photo is rotated, this direction tells us how it was turned.
4. **128-D descriptor** — write 128 numbers that describe the small patch around the point. Two points match if these number lists are close.

Uses: find the same object, stitch photos, 3D rebuild.
Plus: works with zoom and rotation.
Minus: slow / heavy on the computer.

Write this in the paper:
SIFT detects and describes keypoints robust to scale and rotation. Pipeline: Gaussian scale space → DoG extrema → orientation → 128-dimensional descriptor. Strong but computationally heavy. Used in object recognition, stitching and 3D reconstruction.

---

17. SURF (Speeded-Up Robust Features)

Purpose (write this): A faster SIFT. Still finds local features that survive scale and rotation, but it runs quicker.

In simple words: same job as SIFT (find special points + describe them + match them). SURF was made in 2006 to do that job **faster**.

Three parts (from the PDF):

1. **Feature detection** — find interest points (corners, blobs, junctions). It uses a **Hessian** score. You do not need the full maths. Say: *a number that tells how much this point stands out.*
2. **Feature description** — write the small area around the point as **64 or 128 numbers**.
3. **Matching** — compare those number lists between two images. Close lists = the same point.

Compare for the paper: SIFT = stronger but slower. SURF = faster SIFT-like. ORB = even faster, weaker if the object changes size a lot.

Write this in the paper:
SURF is a faster alternative to SIFT. It detects interest points using a Hessian-based measure,

describes each point with a 64 or 128-D vector, and matches those descriptors across images. It is scale and rotation invariant with lower computational cost than SIFT.

---

18. HOG (Histogram of Oriented Gradients)

Purpose (write this): Find which way the edges are pointing (horizontal, vertical, or diagonal) in each small section of the image.

In simple words: SIFT picks a few special dots. HOG looks at a **whole window** (a walking person, a face) and asks: *in this box, which way do the edges point?* A standing person has more up-down edges (legs, body). That pattern is the HOG.

Pipeline (PDF order):

1. **Compute gradients** — find how brightness changes, and in which direction (the edge direction).
2. **Divide into cells** — small squares, e.g. 8×8 pixels.
3. **Orientation histogram** — in each cell, count how many edges point left, up, right, etc.
4. **Normalise over blocks** — join a few cells and scale the numbers so a bright photo and a dark photo still look similar. This helps with lighting.

Uses: pedestrian detection, object classification.
Plus: light, easy to explain.
Minus: if the person is much bigger or smaller than expected, it can fail (**scale**).

Write this in the paper:
HOG captures edge orientation distributions in local cells, then normalises over blocks. Used for pedestrian detection and classification. Lightweight and interpretable, but sensitive to scale changes.

---

19. Stereo Vision

Purpose (write this): Use two images from slightly different viewpoints to estimate **depth** (how far things are).

In simple words: you have two eyes. Each eye sees a slightly different picture. Your brain uses that difference to feel distance. Stereo vision does the same with two cameras.

1. Take a **left** photo and a **right** photo.
2. Find the **same point** in both (a corner of a car, a tree).
3. See how much it **shifted**. Big shift = **near**. Small shift = **far**.

Uses: 3D models, self-driving, robots.
Problem: a blank wall has no unique points, so matching fails (**textureless** regions).

Write this in the paper:
Stereo vision estimates depth by matching corresponding points between left and right images taken from slightly different viewpoints. Used in 3D reconstruction, autonomous driving and robotics. Challenge: unreliable matches in textureless regions.

---

20. Epipolar Geometry

Purpose (write this): The maths that explains how two camera views of the same scene are related.

Two names from the PDF:

- **Essential matrix** — you **know** the camera settings (calibrated).
- **Fundamental matrix** — you **do not** know the camera settings (uncalibrated).

Uses: stereo matching, building 3D from many photos (structure-from-motion), camera calibration.

Write this in the paper:
Epipolar geometry describes the relationship between two camera views. A point in one image must correspond to a point on the epipolar line in the other image. Essential matrix is for calibrated cameras; fundamental matrix is for uncalibrated cameras.

20b. Optical Flow (Lucas–Kanade and Horn–Schunck)

Purpose (write this): Estimate the **apparent motion** of pixels from one video frame to the next.

In simple words: frame 1 has a ball. Frame 2 has the ball a bit to the right. Optical flow draws small arrows: "this pixel moved here."

How it works: it assumes a pixel keeps the **same brightness** while it moves. Then it computes how far it moved (a displacement vector). If the light suddenly changes, this assumption breaks.

Two methods in the PDF:

Method	Simple idea
Lucas–Kanade	Look only in a **small window** around the point. Fast. Used in KLT tracking.
Horn–Schunck	Look at the **whole image** and assume nearby pixels move in a **smooth** way (not random jumps).

Uses: gestures, motion detection, steady video (stabilisation).
Limits: lighting change and hiding (occlusion).

Week 2 also says **sparse** (only some points) vs **dense** (every pixel). That is section 25.

Write this in the paper:
Optical flow estimates apparent motion of pixels between consecutive frames. It assumes brightness constancy. Lucas–Kanade uses a local neighbourhood. Horn–Schunck uses a global smoothness constraint. Limitations: lighting changes and occlusions.

20c. Kalman Filters

Purpose (write this): A recursive method that **predicts** and then **corrects** the position of a moving object over time.

In simple words: Imagine a car moving on the road. The computer detects the car in every video frame, but the detected box may **shake or jump slightly** because of small errors. The car is actually moving smoothly. The Kalman Filter predicts where the car should be next and then corrects its prediction using the new detection. It repeats this for every frame.

Kalman does two steps, again and again:

1. **Prediction** — from how it was moving, guess where it should be next ("a bit further on the road").
2. **Update (correct)** — look at the new measurement (the box) and mix it with the guess. Too jumpy measurement? Trust the guess more. Clean measurement? Trust the box more.

Plus: works with noisy sensors, makes the path smooth.
Uses: cars, drones, people, in real time.

Note: best when motion is **straight-ish (linear)** and noise is **Gaussian** (normal bell-curve noise). If motion is curved / non-linear, use **Extended Kalman Filter**.

Write this in the paper:
A Kalman filter is a recursive algorithm that predicts the next state of a moving object and then corrects it using observed measurements. It handles noisy data and smooths trajectories. Best for linear motion with Gaussian noise; extended Kalman filters handle non-linear cases.

Then continue Week 2 and the syllabus order: challenges → features → Harris/SIFT/ORB/HOG recap → matching → flow in video labs → ML → face → CNN.

21. Four core challenges of Computer Vision

The same real object can look totally different in a photo. That is why CV is hard. Memorise **four words + one example each**.

Challenge	Meaning	Example
Viewpoint	camera angle changes the silhouette	a cup from the top is a circle; from the side it is a U-shape
Illumination	light changes brightness and color	a face in sun vs in a room
Occlusion	part of the object is hidden	a person behind a car
Scale	near looks big, far looks small	the same dog filling the frame vs in the distance

Hand-crafted features (SIFT) and later CNNs are both attempts to survive these changes.

Write this in the paper:
Core CV challenges are viewpoint variation, illumination changes, occlusion and scale variation. A robust system must recognise the same object under these changes.

---

22. What is a “feature”?

You cannot compare two photos pixel by pixel. If the camera moved 2 cm, almost every pixel changed, even if it is the same room.

So we pick **special places** that are likely to be found again: corners, blobs, distinctive textures. Those places are **keypoints**. Around each keypoint we write a list of numbers that describe the neighborhood. That list is the **descriptor**.

- **Keypoint** = *where* (x, y, sometimes scale and orientation)
- **Descriptor** = *what it looks like nearby* (a vector)

A good feature is:

- **Distinctive** — not confused with other places
- **Repeatable** — found again after rotate / scale / light change
- **Compact** — a short vector, not the whole image

Write this in the paper:
A feature is a distinctive, repeatable pattern. A keypoint marks its location. A descriptor encodes

the local neighborhood as a vector so two images can be matched without comparing every pixel. (write about good feature written in above points).

---

23. Harris, SIFT, ORB, HOG

Harris corners

A **corner** is where intensity changes in **two** directions (an edge changes in only one; a flat wall changes in none).

Harris looks at gradients in a window and scores "how corner-like". Good on buildings and chessboards. **Not** invariant to scale or rotation: if you zoom, the window no longer matches.

SIFT (Scale-Invariant Feature Transform)

The classic strong descriptor. Pipeline (learn this order):

1. Build a **Gaussian scale space** (blur the image at many scales)
2. Find **extrema in Difference of Gaussians** (DoG) — those are candidate keypoints
3. Assign an **orientation** from local gradients (so rotation can be undone)
4. Write a **128-dimensional** descriptor of the neighborhood

Strength: survives scale, rotation, a fair amount of light change.
Limit: **slow / heavy**. (It was patented; now free in modern OpenCV.)

ORB (Oriented FAST and Rotated BRIEF)

A **fast, free** alternative. Uses FAST to find corners, BRIEF as a binary descriptor, plus orientation.

Strength: real-time, good enough for panorama / AR on a phone.
Limit: weaker than SIFT when scale change is large.

HOG (Histogram of Oriented Gradients)

Not a sparse keypoint method. It describes a **region** (a whole pedestrian, a face window):

1. Compute gradients
2. Split into cells (e.g. 8×8)
3. In each cell, make a histogram of gradient **directions**
4. Normalise over bigger blocks (helps with lighting)

Used for **pedestrians and faces**. Interpretable and fairly light. Sensitive to **scale** (person must be roughly the expected size).

Write this in the paper:
Harris detects corners but is not scale/rotation invariant. SIFT detects and describes keypoints

robust to scale and rotation using Gaussian scale space, DoG extrema, orientation and a 128-D descriptor; it is computationally heavy. ORB is a faster free alternative. HOG builds histograms of edge directions in cells and is used for pedestrian/face detection.

---

24. Matching, Homography, RANSAC

Step 1: Matching

Compare the descriptors of **Image A** and **Image B** to find the same feature points.

Step 2: Lowe's Ratio Test

Sometimes two matches look similar. Keep a match **only if the best match is much better than the second-best match**. This removes confusing matches.

Step 3: Homography

A **homography** is a **3×3 matrix** that tells how to **align or transform one image onto another**. It is used in panorama stitching and logo detection.

Step 4: RANSAC

Some matches are still wrong. **RANSAC** repeatedly tests different groups of matches and **keeps the transformation that fits the most correct matches**, while ignoring wrong ones (outliers).

That is also how a **logo detector** draws a box in a scene.

Write this in the paper:
Image matching finds correspondences between descriptors. Homography maps one image plane to another. RANSAC estimates that transform robustly by ignoring outlier matches.

---

25. Optical flow and tracking

Optical flow = apparent motion of pixels from one video frame to the next.

Assumption: a pixel's brightness stays the same as it moves (brightness constancy). If lighting flashes, this breaks.

Two flavours:

- **Sparse:** track only some good points (usually corners). Cheap. **KLT / Lucas–Kanade** does this.
- **Dense:** a motion vector at **every** pixel (Farneback). Heavier, nice for visualising "what moved".

Object tracking means follow a target across frames. You can detect every frame (YOLO) and link boxes, or track by motion (optical flow), or both (DeepSORT). Hard cases: occlusion, fast jumps, scale change, many similar objects.

Week 2 adds two practical flavours:

- **Sparse flow:** track only some good points, usually corners. Cheap. **KLT** uses Lucas–Kanade.
- **Dense flow:** a motion vector at **every** pixel (Farneback). Heavier. Good for visualising "what moved".

Write this in the paper:
Optical flow estimates pixel motion between consecutive frames under brightness constancy. Lucas–Kanade is local; Horn–Schunck is global and smooth. Sparse flow tracks selected keypoints (KLT). Dense flow computes motion at every pixel. It is sensitive to lighting changes, large displacements and occlusion.

---

26. Classical ML on images (kNN, SVM, Bag of Visual Words)

Before CNNs, computer unable to extract the feature so we extracted features by hand, then gave a vector to a classifier.

kNN: look at the *k* nearest training examples; majority label wins. Simple, no real "training", slow on big data.

SVM: SVM (Support Vector Machine) is a classification algorithm that finds the best decision boundary (hyperplane) to separate different classes. It chooses the boundary with the largest margin because a larger margin usually gives better classification. SVM performs well when the input features are good.

Bag of Visual Words (BoVW): like a word-count for images.

1. Extract local descriptors (SIFT/ORB) from many images
2. Cluster them (k-means) → each cluster center is a **visual word**
3. For a new image, the computer checks **which visual words are present** and **how many times each one appears**.
4. The histogram is then given to a classifier like **SVM** or **kNN**, which predicts the image's class (e.g., **cat, dog, car**)

Write this in the paper:
kNN classifies by majority vote of nearest neighbours. SVM finds a maximum-margin hyperplane. Bag of Visual Words clusters local descriptors into a vocabulary and represents each image as a histogram of those visual words.

Supervised vs unsupervised (Week 2 Pre-CNN notes)

Supervised learning = learn from examples that already have **answers (labels)**. Input + correct output. Later you predict the output for a new input.

- Classification: spam / not spam, digit 0–9

- Regression: predict a number (house price)

Unsupervised learning = only inputs, **no labels**. The algorithm finds groups or structure by itself.

- Clustering (k-means)
- Dimensionality reduction (PCA)
- Association rules

Fruit basket example: supervised = you tell it "this is an apple". Unsupervised = it groups red-round things together without the word "apple".

kNN and SVM are supervised. k-means inside BoVW is unsupervised.

Write this in the paper:
Supervised learning trains on labelled data to predict labels or numbers. Unsupervised learning finds hidden structure in unlabelled data, such as clusters.

Artificial neuron and ANN (Week 2)

An **artificial neuron** is one small calculator, inspired by a brain cell:

1. Take inputs (x_1, x_2, \ldots)
2. Multiply by **weights** (w_i) (importance)
3. Add a **bias** (b)
4. Pass the sum through an **activation** (ReLU, sigmoid, tanh) so the neuron can say "fire / don't fire"
5. Send the output to the next layer

[z = \sum_i w_i x_i + b, \quad output = f(z)]

A **neural network** stacks neurons into **input → hidden → output** layers.

- **Forward pass:** data flows forward, you get a prediction
- **Loss:** how wrong the prediction is (cross-entropy for classes, MSE for numbers)
- **Backpropagation:** send the error backward and update weights

Fully connected nets treat every pixel independently and need too many weights. That is the **bridge to CNNs**: convolution keeps local structure.

Write this in the paper:
An artificial neuron computes a weighted sum of inputs plus bias, then applies an activation. An ANN stacks these units. Forward pass predicts; backpropagation updates weights. CNNs replace huge fully connected layers on images with shared local filters.

27. Face detector (Module 5) — Haar vs HOG

First, the basic idea

Face detection = finding *where* a face is in an image (drawing a box around it).
It does **not** tell you *whose* face it is. That's face recognition, which is a different step.

Detection always comes first, then recognition.

Haar Cascade

What it looks at: simple patterns of light and dark areas.
Example: the eye region is usually darker than the cheeks, and the nose bridge is brighter than the eyes. Haar features check for these light/dark patterns.

How it works:

- The image is scanned in small windows.
- Each window goes through a series of quick checks (the "cascade").
- If a window fails an early check, it's thrown out immediately as "not a face."
- Only windows that pass all checks are called faces.
- **AdaBoost** is the method that picks which checks are most useful.

Why it's fast: most of the image is clearly not a face, so it gets rejected in the first few cheap steps.

Good at: faces looking straight at the camera, in normal lighting.
Bad at: tilted or side faces, partly covered faces (mask, hand), strange lighting.

HOG (Histogram of Oriented Gradients)

What it looks at: the direction of edges (gradients) in the image, so it captures the **shape/outline**, like the head and shoulders.

Why it's better: it doesn't depend much on brightness, so it handles different lighting better than Haar.

Downside: slower and needs more computing power.

Quick comparison

	Haar	HOG
Looks at	Light/dark patterns	Edge directions (shape)
Speed	Fast	Slower
Lighting	Weak	Better
Best for	Front-facing faces	Faces and pedestrians

Write this in the paper:
Haar-cascade detection uses rectangular Haar features selected by AdaBoost and a cascade classifier. It is fast for frontal faces but sensitive to pose and lighting. HOG captures gradient orientation histograms, so it is more robust to illumination but computationally heavier.

---

28. Why CNN? Stride, padding, pooling, flatten

Why CNN? (the problem it solves)

A normal neural network (fully connected) has two problems with images:

1. **Too many numbers.** A 200×200 color image = 120,000 inputs, which means huge weight matrices.
2. **Forgets layout.** It treats every pixel as separate, ignoring that nearby pixels belong together.

CNN fix: it uses a small filter (kernel) that slides across the whole image and **reuses the same weights everywhere**. This is called **weight sharing**. It means fewer parameters and the image layout is kept.

Key terms (easy definitions)

Term	Simple meaning
Kernel / Filter	A small grid of numbers that slides over the image to detect one pattern (edge, texture, etc.)
Feature map	The output of one filter sliding over the image. It shows *where* that pattern appears and how strongly. One filter = one feature map.
Stride	How many pixels the filter jumps each step. Stride 1 = moves one pixel at a time. Stride 2 = skips, so the output is about half the size.
Padding	Extra border (usually zeros) added around the image. Without it, the image shrinks after every convolution and edge pixels get used less. "Same" padding keeps the size unchanged.
Pooling	Shrinks a feature map by summarizing small windows. **Max pool** keeps the largest value (strongest detection). **Average pool** keeps the mean. Benefits: smaller size, less computation, tolerates small shifts.
Flatten	Reshapes the 3D block (height × width × channels) into one long 1D list, because fully connected layers need a 1D input. It's only a reshape, no learning.
ReLU	Activation function: negative values become 0, positive values stay. It adds **non-linearity** (without it, many layers would act like just one layer). It is also faster than sigmoid and helps with vanishing gradients.

Typical CNN flow

Input → Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → Fully Connected → Class scores

mnemoics: CRP twice and FFC(once)

- **Early layers** learn simple things: edges, blobs.
- **Deeper layers** learn bigger things: object parts, then whole objects.

Simple way to remember: *Conv extracts, Dense decides.*

Fully connected vs Convolutional

	Fully connected	Convolutional
Connections	Every neuron connects to every input	Only to a small local area
Input	Flattened, so layout is lost	Keeps height × width × channels
Job	Combine features for the final decision	Extract edges, textures, shapes
Parameters	Very many	Few (filter is shared)
Bonus	None	Finds a pattern at any position

Write this in the paper:
CNNs use convolution so filters detect local patterns with few parameters. A feature map is the response of one filter across the image. Stride is the kernel step size. Padding preserves border information and controls output size. Pooling downsamples feature maps (max or average). Flatten converts maps to a vector for the classifier. Fully connected layers learn global combinations but ignore spatial layout; convolutional layers preserve space and share weights.

STEP 3 — Deep CV tasks (Week 3)

Module 3 + Module 4. Highest exam weight.

3A — Architectures + transfer learning (must)

---

29. The architecture lineage (very likely long answer)

The big picture

Each architecture was built to **fix a problem** from the one before it. If you remember the problem, you remember the model.

Story in one line: LeNet proved CNNs work → AlexNet made them big and fast → VGG made them deep and simple → ResNet made very deep nets trainable → EfficientNet made them efficient.

---

1. LeNet (1998), Yann LeCun

- **What it is:** a very small CNN: conv → pool → fully connected.
- **Trained on:** MNIST (handwritten digits).

- **Used for:** reading numbers on bank cheques.
- **Problem it solved:** showed that CNNs work better than older methods on images. It was a **proof of concept**.

---

2. AlexNet (2012)

- **What it is:** 5 conv layers + 3 fully connected layers, trained on GPUs.
- **Result:** won the ImageNet competition by a huge margin. After this, everyone started using deep CNNs.
- **Three ideas that stayed:**
- **ReLU** (instead of sigmoid/tanh): training is faster and vanishing gradient is reduced.
- **Dropout:** randomly switch off some neurons during training so the network can't just memorise the data.
- **GPU:** made it possible to train such a big network.
- **Problem it solved:** showed that deep networks can work at large scale.

---

3. VGG (2014)

- **What it is:** very deep (16 or 19 layers) but a simple design. It just stacks many small **3×3 filters**.
- **Why small filters?** Two 3×3 layers cover about the same area as one 5×5 filter, but with **fewer parameters** and **more non-linearity** (more ReLUs).
- **Problem it solved:** how to go deeper without a messy design.
- **Limitation:** still very heavy (a huge number of parameters).

---

4. ResNet (2015)

- **The problem:** if you keep adding layers, training breaks. Gradients vanish and accuracy can actually get *worse*.
- **The fix: skip (residual) connections.** The input is copied forward and added to the layer's output: **output = F(x) + x** Here, F(x) is what the layer learns (the "change" to make), and x is the original input passed along.
- **Why it works:** gradients can flow easily through the skip path, so very deep networks (50, 101, 152 layers) can be trained.
- **Used for:** transfer learning and medical imaging (very popular).
- **Problem it solved:** vanishing gradients and the drop in accuracy in very deep networks.

5. EfficientNet (2019)

- **The problem:** earlier models improved by only going deeper, or only wider, or only using bigger images. That wastes computation.
- **The fix: compound scaling.** Increase depth, width and image resolution **together**, using one scaling factor.
- **Result:** high accuracy with fewer parameters. Works well on phones and small devices.
- **Problem it solved:** unbalanced and inefficient scaling.

Transfer learning (Module 4)

Idea: don't train from zero. Reuse a model that is already trained on a huge dataset (like ImageNet).

A model like ResNet has already learned to see edges, textures and shapes. For your small dataset (for example, hospital images):

1. **Freeze the early layers.** They hold general features (edges, textures), so you keep them as they are.
2. **Replace and train the last layers** on your own data.

Benefit: needs less data and less time than training from scratch.

Quick revision table

Model	Year	Key idea	Problem solved
LeNet	1998	Small conv-pool-FC net	Proved CNNs work
AlexNet	2012	ReLU, Dropout, GPU	Deep nets at large scale
VGG	2014	Many 3×3 filters, simple and deep	Going deeper cleanly
ResNet	2015	Skip connection (F(x) + x)	Vanishing gradients in deep nets
EfficientNet	2019	Compound scaling	Wasteful scaling

Memory tricks

- **Order: L**ove **A**ll **V**ery **R**eal **E**xams (LeNet, AlexNet, VGG, ResNet, EfficientNet)
- **AlexNet = "RDG": R**eLU, **D**ropout, **G**PU

- **VGG = small filters, big depth**
- **ResNet = skip path.** Think of it as a shortcut so gradients don't get lost.
- **EfficientNet = scale everything together**

Write this in the paper:
LeNet proved CNNs on digits. AlexNet introduced ReLU, dropout and GPU training and started the deep-learning boom. VGG showed depth with stacked 3×3 filters. ResNet's skip connections solved vanishing gradients so very deep nets could train. EfficientNet's compound scaling balances depth, width and resolution. Transfer learning reuses a pre-trained backbone and adapts the last layers to a new task.

---

30. Dropout and skip connections (two easy extra marks)

Dropout — purpose: stop the network from memorising (overfitting).

In simple words: while training, randomly switch off some neurons (say 30%). They output 0 for that step. The network cannot depend on one "star" neuron. Everyone must learn. When you *test*, dropout is **off** — all neurons work.

Think of a team: random players sit out in practice, so the whole team knows the game.

Skip connections — purpose: a shortcut. The input is copied forward and added later: **output = F(x) + x**.

They appear in **two** models. The shortcut is the same idea, the *reason* is different:

	ResNet skip	U-Net skip
Why	so a **very deep** net can still train	so **edges / small details** are not lost
What it helps	**gradient** can flow back (vanishing gradient)	decoder gets the original sharp pixels from the encoder

Write this in the paper:
Dropout is regularisation: random neurons are set to zero during training so the model does not overfit. At test time dropout is off. Skip connections copy input forward. In ResNet they help gradient flow so deep nets can train. In U-Net they copy high-resolution features to the decoder for sharp masks.

3B — Detect and segment (must)

31. Four vision tasks (do not mix them)

Same photo. Four different jobs. Do not mix the names.

Task	What the computer must say	Example
Classification	one label for the **whole** photo	"this is a cat"
Object detection	**box + name** for each object	"cat here, dog there"
Semantic segmentation	a name for **every pixel**; two cars get the **same** label	road / car / sky map
Instance segmentation	every object separately, even two cars	car-1 mask, car-2 mask

Generation (GAN / diffusion): **GANs** and **Diffusion Models** do **not** identify or label an existing image. Instead, they **learn patterns from many training images and generate a brand-new image** that looks realistic. GANs and Diffusion Models are popular image generation techniques.

Write this in the paper:
Classification gives one label for the image. Detection gives a box and class for each object. Semantic segmentation labels every pixel (same class = same colour). Instance segmentation separates each object. Generation creates a new image.

---

32. Object detection: R-CNN family, YOLO, SSD

Purpose: find **what** is in the image and **where** (a box).

In simple words: classification says "there is a dog". Detection draws a rectangle around each dog.

R-CNN (2014) - two stage

Name = Region-based CNN.

1. **Guess regions.** First, propose many possible boxes where an object might be (around 2000 per image).
2. **Run a CNN on each box.** Crop each box and pass it through a CNN separately to extract its features.
3. **Classify and fix the box.** Decide what the object is (cat, dog, etc.) and adjust the box so it fits the object better.

Plus: accurate. **Minus:** very slow (thousands of CNN runs). Fine for a doctor looking at a scan later. Bad for a car that must decide now.

Faster R-CNN: a small net called **RPN (Region Proposal Network)** learns the boxes. Still two stages: first propose, then classify.

YOLO (2016) — You Only Look Once

Looks at the **whole** image **once**. In that one pass it predicts all boxes and all classes together.

Plus: real-time (fast). **Minus:** weak on **tiny** objects (a far person can be missed). Use for driving, sports, live CCTV.

SSD (2016) — Single Shot MultiBox Detector

SSD is an object detector that works in one shot, which means it is a single-stage method and not a two-stage one like R-CNN. It looks at the whole image once and predicts the objects and their boxes together, so there is no separate step for guessing regions first. To handle objects of different sizes, it makes predictions from several feature maps of different sizes. The larger maps keep more detail and help catch small objects, while the smaller, deeper maps see the big picture and help catch large objects. In terms of performance, SSD sits in the middle: it is faster than R-CNN-type detectors but usually a little less accurate than the slowest two-stage models. Because it is light and fast enough, it is commonly used on phones and other small devices.

They love this choice question:

- Medical / tumour → need **accuracy** → R-CNN family
- Self-driving / live camera → need **speed** → YOLO (or SSD)

Write this in the paper:
R-CNN classifies region proposals with a CNN: accurate but slow. Faster R-CNN adds an RPN to propose regions. YOLO predicts boxes and classes in one forward pass: real-time, weaker on small objects. SSD is also single-shot and uses multi-scale feature maps. Choose R-CNN when accuracy matters more than speed; YOLO/SSD when real-time is required.

---

33. Semantic segmentation: FCN and U-Net

Purpose: give **every pixel** a class. Not one label for the photo. Not a box. The actual shape.

In simple words: classification = “cat”. Segmentation = colour every cat pixel.

FCN (Fully Convolutional Network)

Old CNNs ended with fully connected layers. Those layers **flatten** the picture, so “where” is lost.

FCN **removes the dense layers** and uses **only convolution layers**. Because convolutions keep the spatial layout, the output is a **map shaped like the image**, where each pixel gets a class label.

Plus: first big idea for pixel labels. **Minus:** the mask is **rough**. Edges of the tumour look fuzzy.

U-Net (2015)

Draw this in your head:

```
Encoder (down):  picture gets smaller → understands WHAT it is
Decoder (up):    picture grows back   → understands WHERE it is
Skip connections: copy encoder details into decoder → sharp edges
```

- Encoder = compress for **meaning** (this looks like tissue).
- Decoder = expand for **place** (these pixels).
- Skips = bring back the original edges that compression lost.

That is why U-Net is used in **medical** images: you need the exact boundary, not a box.

Write this in the paper:
Semantic segmentation labels every pixel. FCN replaces fully connected layers with convolutions so the output is a dense pixel map, but boundaries can be coarse. U-Net is an encoder-decoder with skip connections: the encoder captures context, the decoder recovers location, skips bring fine detail. This is why U-Net is standard in medical imaging.

---

34. Instance segmentation: Mask R-CNN

Start with the problem

A bounding box only says "the object is somewhere in this rectangle." But a rectangle also includes background. A tumour box includes healthy tissue. Two people standing close have overlapping boxes. So we need something more exact: the **real shape** of the object. That is what a **mask** gives.

How Mask R-CNN works, in order

Step 1: Start with Faster R-CNN.
Faster R-CNN already finds objects. First, a small network called the **RPN (Region Proposal Network)** suggests boxes where objects might be. These suggestions are called **proposals**.

Step 2: Cut out features for each proposal using RoI Align.
For each proposal, the network takes that part of the feature map. **RoI Align** does this cutting carefully, without rounding the coordinates, so the cut-out lines up exactly with the object. (The older RoI Pool rounded the numbers, so the mask sat slightly off.)

Step 3: Predict three things for each object.

- **Box:** where it is
- **Class:** what it is

- **Mask:** which exact pixels belong to it

The **mask branch** is the new part. It is the extra branch added on top of Faster R-CNN, and it draws a black-and-white mask for each object.

Result

Precise shape for every object, even overlapping ones, which boxes cannot give.

Exam-style version (direct)

Mask R-CNN extends Faster R-CNN by adding a mask branch. The RPN first proposes regions, and RoI Align then extracts aligned features for each one without rounding. For every object, the network outputs a bounding box, a class label and a pixel-level mask. The mask gives the precise shape of the object, which a bounding box cannot provide.

3C — Class extras (in notes + question PDF, not a named syllabus bullet)

35. Video: CNN + LSTM (class extra — still in the question PDF)

Purpose: understand **video**, not one photo. Video is photos **in order**.

In simple words: one photo of a hand is not a "wave". A wave is the hand moving **across frames**.

- **CNN** on each frame → **what** is in that picture (a hand, a ball).
- **LSTM** on the sequence of those CNN results → **how it changes** (the wave, the throw).

RNN (simple sequence net): it has a small memory of recent steps. On a **long** video that memory fades. This is **vanishing gradient**.

LSTM (Long Short-Term Memory) is an RNN with **gates** so it can remember longer:

- **Input gate** — what new information to keep
- **Forget gate** — what old information to throw
- **Output gate** — what to send to the next step

LSTM can also handle short and long videos.

Pipeline: CNN per frame → list of feature vectors → LSTM → action class (clap, throw, walk).

Write this in the paper:
A CNN sees what is in each frame (spatial features) but not time. An RNN models sequence but forgets long videos (vanishing gradient). LSTM uses input, forget and output gates to keep long-term memory. CNN + LSTM is used for action and gesture recognition.

---

36. Vision Transformer (ViT) and attention (class extra)

Purpose: a CNN looks at a **small window**. A Vision Transformer looks at the **whole image at once** using **attention**.

In simple words: CNN is a magnifying glass. To connect the zebra's head to its tail you need many layers. Attention lets far parts **talk in one step**.

Attention (came from language models). Three names:

- **Query (Q)** — what I am looking for
- **Key (K)** — what each piece is
- **Value (V)** — the information inside that piece

Compare Q with K → get importance scores → mix the Values with those scores. Related pieces get high weight.

ViT steps:

1. Cut the image into **patches** (e.g. 16×16). Flatten each patch. Treat it like a **word**.
2. Send this list of patches into a **Transformer** (attention).
3. A special **class token** summarises the whole image for the final label (cat / dog).

Patches keep local detail. Attention across patches gives **global** view. ViT usually needs **a lot of data**. Some models mix CNN + Transformer.

Write this in the paper:
Attention uses Query, Key and Value to decide which parts of the input matter and captures long-range relations. ViT splits an image into patches, treats them as tokens, and runs a transformer encoder. A class token is used for classification. This is how ViT sees global context that local CNN filters miss.

STEP 4 — GANs (generative models)

Module 6, and "image generation" in Module 3.

---

37. GAN — two networks in a fight

Start with the purpose

A normal **classifier** looks at an image and says "this is a 7." A **GAN** does the opposite job: it **draws a brand-new 7** that was never in the dataset. That is why it is called a **generative** model (it generates new data).

GAN = Generative Adversarial Network (Ian Goodfellow, 2014).

The idea: a forger and the police

A GAN has **two networks with opposite jobs**. Think of a fake-money maker and a police officer.

Network	Role	Input	Output
Generator (G)	The forger	Random numbers (noise, called **z**)	A fake image
Discriminator (D)	The police	A real or fake image	A score from 0 to 1 (how real does it look?)

Important: G never sees or copies real photos. It learns only from D's feedback ("fake" or "real"). D, on the other hand, sees both real images and G's fakes.

How training goes, step by step

1. **Start:** G knows nothing, so it makes noise. D easily catches every fake.
2. **Middle:** G uses D's feedback and gets a little better. Now D has to look more carefully, so D also improves.
3. **Success:** G's pictures look real. D cannot tell the difference and can only guess, so its score is about **0.5** (50-50).

After training: we keep **G** (the artist) and throw away **D** (it was only a teacher).

The three words you must write: adversarial, zero-sum, minimax

- **Adversarial:** the two networks are opponents.
- **Zero-sum:** D wants to be right, G wants D to be wrong. One's win is the other's loss.
- **Minimax:** this fight is the objective of the GAN. D tries to maximise its correctness while G tries to minimise it.

Memory tricks

- **G = Generator = Gets noise, Gives fake.**
- **D = Discriminator = Decides real or fake.**
- **Forger vs Police.**
- **Success = D score about 0.5.**

- **Keep G, drop D.**

Exam-style version (direct)

A GAN has a generator that turns random noise into images and a discriminator that scores images as real or fake. The two networks train as opponents with a minimax objective: the generator tries to fool the discriminator, and the discriminator tries to catch the fakes. When training succeeds, the generated images look like real data and the discriminator outputs about 0.5. After training, the generator is kept and the discriminator is discarded.

---

38. Nash equilibrium, instability, mode collapse

Nash Equilibrium

A **Nash Equilibrium** is the ideal point where **both the Generator (G) and Discriminator (D) are as good as they can be**. The Generator creates images that look real, and the Discriminator **cannot tell real from fake**, so it guesses **50% real, 50% fake (0.5 probability)**.

Why don't GANs reach it?

During training, **G and D keep improving against each other**. As soon as one improves, the other adapts. So instead of settling at one point, they often **keep chasing each other**, making training unstable.

Mode Collapse

Sometimes the Generator discovers **one type of fake image** that fools the Discriminator. Instead of creating many different images, it **keeps generating the same (or very similar) image repeatedly**. The images may look realistic, but **there is no variety**.

Training Instability

GAN training is often **unstable**. The loss may go up and down, and sometimes **training longer makes the generated images worse instead of better**. Therefore, **do not judge a GAN only by its loss value**. Instead, look at the **generated images** to see if their quality is improving.

Common fixes (just remember the names)

- **Label smoothing**
- **Instance noise**
- **Different learning rates** for Generator and Discriminator

- **Minibatch statistics** (helps prevent mode collapse by encouraging image variety)

Write this in the paper:

GAN training is a game between the Generator (G) and the Discriminator (D). The ideal goal is a Nash Equilibrium, where G generates realistic images and D cannot distinguish real from fake (outputs 0.5). In practice, training often oscillates instead of converging. Mode collapse occurs when G repeatedly generates only a few types of images, reducing diversity. GAN training is also unstable, so loss values alone do not reflect image quality; the generated images should also be evaluated.

---

39. DCGAN and Conditional GAN

Same fight as section 37. Only the **network design** (and extra label) changes.

A **DCGAN** is simply a **GAN that uses CNNs (convolutional layers)** instead of only fully connected layers. This helps it understand image patterns much better.

- The **Discriminator (D)** uses **strided convolutions** to gradually **reduce the image size** while extracting features and deciding whether the image is real or fake.
- The **Generator (G)** uses **transposed convolutions** to gradually **convert random noise into a full-size image**.
- **Batch Normalization (BatchNorm)** is often used to make training more stable.

Result: DCGAN generates **sharper and more realistic images** than a basic GAN.

Conditional GAN (cGAN)

A **Conditional GAN (cGAN)** is a GAN where **both the Generator and the Discriminator are given a class label** (for example, **"7"**).

This lets you **control what the Generator creates**.

Example:

- Label = **7** → Generate an image of the digit **7**.
- Label = **Cat** → Generate a **cat**.

Without a label, the Generator creates **any random image** it has learned.

Exam Tip

> **DCGAN is a GAN that uses convolutional layers. The Discriminator uses strided convolutions to extract features, while the Generator uses transposed convolutions to generate images, producing sharper results. A Conditional GAN (cGAN) also receives a class label, allowing it to generate a specific type of image instead of a random one.**

---

40. Diffusion models (syllabus says “overview” — not in your slides)

How does a Diffusion Model generate images?

A diffusion model first **learns by destroying images**, and later **generates images by rebuilding them**.

1. **Training (Forward Diffusion):** It starts with a real image and **adds small amounts of noise again and again** until the image becomes **pure random noise**. At each step, the model learns **how the noise was added**.
2. **Generation (Reverse Diffusion):** To generate a new image, the model starts with **pure random noise**. It then **removes a little noise at a time**, gradually revealing meaningful patterns. After many steps, the random noise is transformed into a **new realistic image**.

Purpose: another way to **make images**.

In simple words: start from **TV static** (pure noise). Learn to **remove a little noise**, many times, until a clear picture appears. No forger vs police.

	GAN	Diffusion
How	two nets fight	slowly denoise
Stability	often unstable, mode collapse	usually more stable
Control	cGAN uses a class label	can also follow text
Today	still taught	what most “AI image” apps use

Four or five lines in the paper is enough unless the whole question is generation.

Write this in the paper:
Diffusion models generate images by starting from noise and removing noise step by step. They are generally more stable than GANs. GANs use an adversarial generator-discriminator game. Diffusion is now common for high-quality image generation.

41. Multimodal AI (in the subject name, almost no lecture)

Purpose: one model that understands **more than one type of data** — usually **image + text**.

Unimodal = only pictures, or only words.
Multimodal = pictures **and** words together.

CLIP idea (simple): train so a photo and its **correct caption** sit **near** each other in one shared number space. A wrong caption sits far. Then you can:

- search photos by typing words
- classify without a fixed list of class names
- go toward captioning / vision-language models

A normal CNN: image → class id.
A multimodal model: **image and text into the same space**.

If the paper never asks this, you lose nothing. If one short question appears, the paragraph below is enough.

Write this in the paper:
Multimodal AI understands more than one data type, typically vision and language. Models like CLIP put images and text in a shared space so you can search or classify images with natural language.

STEP 5 — Questions, then project

Practice files (same as STUDY-ORDER.md):

- 01-Week1-Questions.pdf → answers: ANSWERS-Week1-Questions.md
- 02-Week2-and-Week3-Questions.pdf → answers: ANSWERS-Week2-and-Week3-Questions.md
- CV Mid Term Sample Questions.pdf → answers: ANSWERS-CV-Mid-Term-Sample-Questions.md
- 03-CV-Project-Assignment.pdf — project brief only (not answered here)

Quick compare table (last-night revision)

If they ask	Answer in one breath
RGB vs HSV	RGB for screens; HSV for color detection under light change
Global vs adaptive vs Otsu	one T vs local T vs auto T by variance
Opening vs closing	erode→dilate (noise) vs dilate→erode (holes)
Harris vs SIFT vs SURF vs ORB	corners / strong slow / faster SIFT-like / fastest cheap
Lucas–Kanade vs Horn–Schunck	local window vs global smoothness
Dense vs fully connected	local shared filters vs every-to-every, layout lost
Haar vs HOG face	fast frontal vs slower, better lighting
Classification vs detection vs segmentation	label / box / every pixel
Semantic vs instance	all cars same vs each car separate
R-CNN vs YOLO	accurate slow vs real-time
FCN vs U-Net	first pixel-map vs encoder-decoder + skips
ResNet skip vs U-Net skip	train deep vs restore edges
RNN vs LSTM	short memory vs gated long memory
CNN vs ViT	local filters vs global attention on patches
GAN vs diffusion	fight two nets vs denoise noise
DCGAN vs cGAN	conv architecture vs add a class label

Suggested use with internship

Sitting	Read in this file
1	Sections 1–7 (pixels to edges)
2	Sections 8–12 (histogram to contours) then Week 1 questions
3	Sections 13–20c (tracking → SIFT/SURF/HOG → stereo/epipolar → flow → Kalman)
4	Sections 21–28 (challenges, features, matching, ML, face, CNN words)
5	Sections 29–30 (lineage) — write it from memory
6	Sections 31–34 (tasks, detect, segment)
7	Sections 35–36 if time (video, ViT)
8	Sections 37–41 (GAN + extras) then Week 2–3 questions

After each sitting, write 8 lines **without looking**. That is the exam skill.

When you want the original teacher PDF, open the matching link in STUDY-ORDER.md. You do not need those PDFs first if you actually understand this file.